D1 Detailed Design: Smart Barbell Load Cuffs

Assignment 4, Part 3 · Design D1
Team: Tyler Beck, Colin Hill
Source files (this folder): `figures/D1_Data_Model.mmd` (Mermaid source), `figures/build_figures.py` (renders and frames every figure), `D1_Data_Model.png`, and this document. D0 is unchanged beside it.

1. Header, scope, and conventions

Title: Smart Barbell Load Cuffs

Goal statement: Log every set's weight, reps, and bar speed to the right lifter's account automatically, starting from one phone tap on the bar.

Scope: This document details **C5 Account service** (it holds the data model and the API every other component calls) and **C2 Ring firmware** (its weight and rep algorithms are the computations that break the product if they are slow or wrong, and it owns the Bluetooth interface I6). C1 Load sensing, C3 NFC sign-in tag, C4 Lifter phone app, C6 Staff dashboard and the glue components (authentication, notifications, configuration, logging) are deferred to D2.

Conventions used in every D1 figure and table

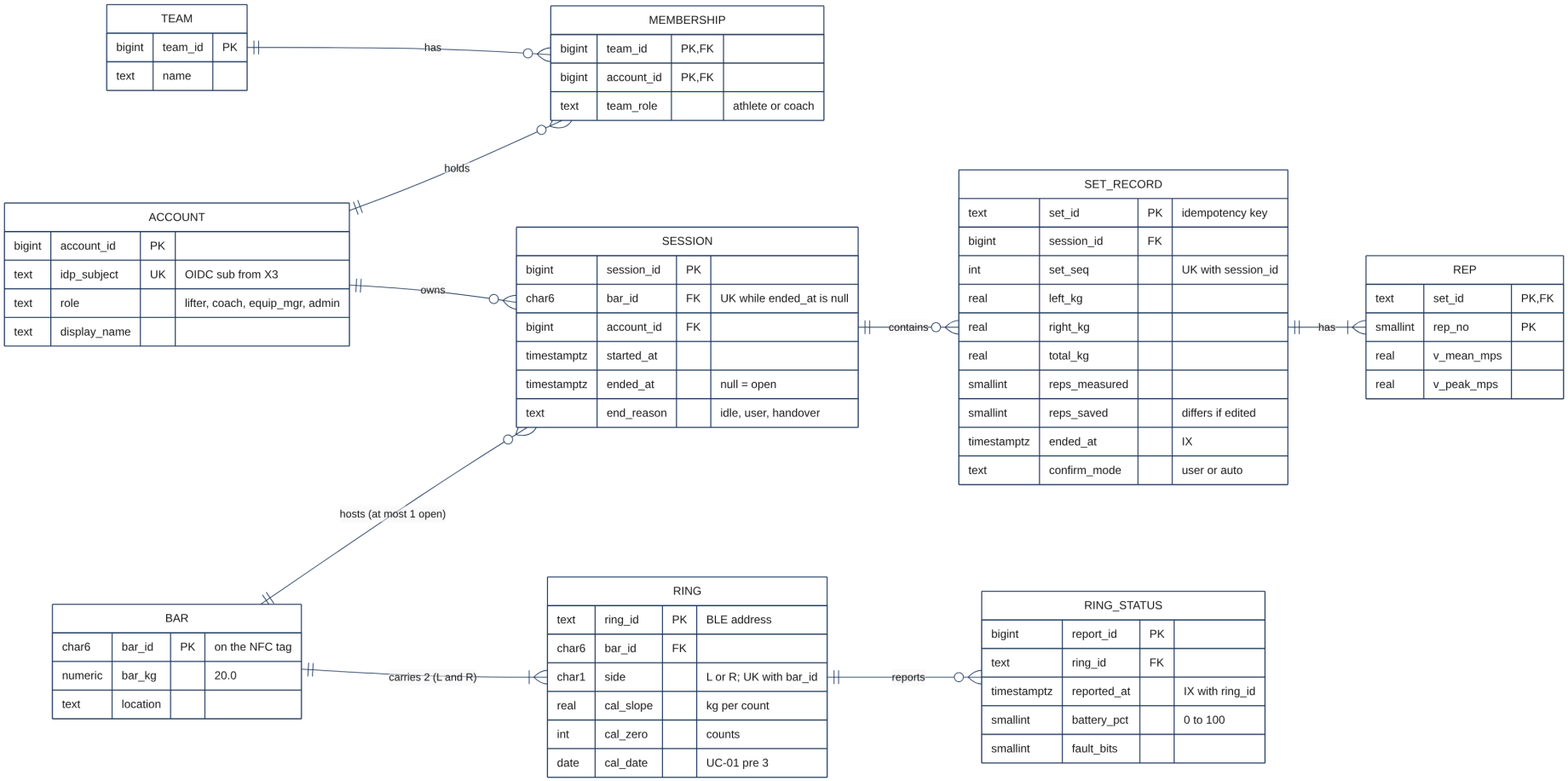
Item	Meaning
Box in Figure 1	One entity, stored as one table. Rows list only the attributes that drive behavior, as *type · name · key · note*.
PK, FK, UK, IX	Primary key, foreign key, unique constraint, index. "UK with bar_id" means a composite unique key; "UK while ended_at is null" means a partial unique index.
Line	A relationship, labeled with a verb read from the entity at the "one" end (SESSION *contains* SET_RECORD).
Cardinality	Crow's-foot notation drawn on both ends: `||` exactly one, `o|` zero or one, `|{` one or more, `o{` zero or more.
C#, X#, I#	Component, external system and interface IDs from D0.
US-##, UC-##, AC-##.#, DL-#	User story, use case, acceptance criterion (User_Stories.md) and D0 decision-log entry.
Types in algorithm and API sections	`int24`, `uint16`, `float32`, … are exact wire or register types; JSON types are `string`, `integer`, `number`, `boolean`; times are ISO-8601 UTC strings; weights in kg, speeds in m/s.

2. Data model, the D1 diagram

Smart Barbell Load Cuffs · D1 · Figure 1 · C5 Account service data model

Team: Tyler Beck, Colin Hill

Goal: log every set's weight, reps, and bar speed to the right lifter's account automatically, starting from one phone tap on the bar.



Conventions

Box	One entity (one table). Rows list only the attributes that drive behavior: type, name, key, note.	**PK / FK / UK / IX**	Primary key, foreign key, unique constraint, index. A note such as 'UK with bar_id' means a composite unique key.
Line	A relationship, labeled with a verb read from the entity at the "one" end (e.g. SESSION contains SET_RECORD).	**Crow's foot ends**	|| exactly one o| zero or one |{ one or more o{ zero or more. Both ends are always drawn.

What it stores. One rec center has about 30 bars (60 rings) and a few hundred lifters. At roughly 50 sets per bar per day that is about 1,500 SET_RECORD rows and 9,000 REP rows a day, or about 0.5 million sets and 3 million reps a year. RING_STATUS adds one row per ring per connection, about 2,000 a day.

Structural decisions

#	Decision	Why	Requirement
S1	**SESSION is an entity**, not just an `account_id` column on each set.	Ownership of a bar is a thing with its own lifetime (start, end, end reason). The rule "at most one open session per bar" needs a row to constrain, and every set inherits exactly one owner through its session.	US-04, UC-02, AC-02.1 to AC-02.4
S2	**REP is an entity**, not a JSON array inside SET_RECORD.	The coach view lists, filters and sorts per-rep velocity across athletes. One row per rep keeps that a plain indexed query. Cost: about 6 extra rows per set, which is negligible at our size.	US-02, I11
S3	**RING is an entity separate from BAR.**	Calibration and battery are per side, and a ring can be moved to another bar or replaced without changing the bar's ID on its tag.	UC-01 pre 2 and 3, US-03
S4	**BAR to RING is one-to-many (exactly 2 in practice)**, enforced by `UNIQUE (bar_id, side)`.	Two columns `left_ring`, `right_ring` on BAR would work, but then a spare ring could not be registered and calibration history would need a third table anyway.	US-03
S5	**RING_STATUS is a time series**, not battery columns on RING.	US-03 needs "last report time" and the D0 rule "not reported for 7 days → check bar" (I12). History also shows battery trends.	US-03, I12
S6	**MEMBERSHIP is many-to-many** between ACCOUNT and TEAM.	An athlete can be on two teams (varsity and club), and a coach can run several. `team_role` lives on the link because a person can coach one team and lift on another.	US-02, I11
S7	**One ACCOUNT table with a `role` attribute**, not separate LIFTER and STAFF entities.	Everyone signs in the same way through X3, and staff also lift. A role check per request (403 on I11) is simpler than two account types.	US-02, US-03, DL-7
S8	**`set_id` is a client-built text key** `barId-sessionId-setSeq`, not a server sequence.	A set can be uploaded twice after a Bluetooth or network drop. A key the phone builds from values the ring already stamped lets the server answer "already stored" instead of creating a duplicate.	UC-01 E1.3, AC-01.4
S9	**`reps_measured` and `reps_saved` are both kept.**	The lifter may correct the count; the measured value is kept for accuracy analysis.	UC-01 A2.2

Indexes and constraints

Index or constraint	Why
`UNIQUE (bar_id) WHERE ended_at IS NULL` on SESSION	The database itself refuses a second open session on a bar, even if two phones race (I10 → 409).
`(account_id, started_at DESC)` on SESSION	Lifter history page, newest first (UC-01 postcondition).
`UNIQUE (session_id, set_seq)` on SET_RECORD	Second guard against duplicates from the ring's 20-set buffer (UC-01 E1).
`(ended_at)` on SET_RECORD	Dashboard date-range queries (I11).
`(ring_id, reported_at DESC)` on RING_STATUS	Latest status per ring for the bar list (US-03).
`UNIQUE (idp_subject)` on ACCOUNT	One account per identity-provider subject; looked up on every request.
`(account_id)` on MEMBERSHIP	"Which teams does this coach see" on every dashboard request.

Relational, not a simpler store. We use a relational database (PostgreSQL). The two rules that protect US-04 (one open session per bar, no duplicate sets) become a partial unique index and a primary key that hold inside one transaction, so no application code can get them wrong under concurrency. The coach and equipment views are joins across four tables. A document store such as Firestore has no partial unique index, so the one-open-session rule would move into application transactions, and the coach query would need denormalized copies of every set. The data is small and highly structured, which is the case relational databases handle best.

3. Core algorithms

Two computations break the product if they are slow or wrong. Both run on C2, once per ring.

A1. Side weight from strain (C2)

1. What it is and the problem it solves. A1 turns the bridge amplifier's raw counts into the weight hanging outside the gauges on one side, and reports it only when the reading can be trusted. The gauge signal is tiny (about 0.73 µV per kg at the bridge), so a few microvolts of drift or a plate clanking on the sleeve would otherwise show up as kilograms. A1 must deliver the side weight within the ±1.0 kg total budget of AC-01.1, and within 1 s of the bar going still (UC-01 step 4).

2. Inputs and outputs

Direction	Name	Exact type	Source and rate
In	`raw_count`	`int32` (sign-extended `int24`)	HX711 over I2, 80 samples/s
In	`pol`	`int8`, +1 or −1	Excitation polarity C2 set before that sample (EXC_POL line)
In	`accel`	`int16[3]`, mg	Motion sensor over I3, 104 samples/s
In	`gyro`	`int16[3]`, mdps	Motion sensor over I3, 104 samples/s
In	`temp_c`	`float32`, optional	Bar thermistor; absent in the sprint build
In	`cal`	`{slope: float32 kg/count, zero: int32 counts, k_temp: float32 counts/°C, t_ref: float32 °C}`	Calibration record in ring flash
Out	`WeightEvent`	`{side: char 'L' or 'R', kg: float32 0.0–200.0, still: uint8, set_seq: uint16}`	I6 notify, at most once per 1 s still window
Out	fault bits	`SENSOR_FAULT`, `OVER_RANGE`, `ZERO_DRIFT` in `status.faults: uint8`	I6 status event

Steps

1. **Cancel drift with excitation reversal.** C2 flips the bridge polarity every sample. For each pair, $d = (c_+ - c_-) / 2$. The real signal flips sign with the excitation; amplifier offset and thermocouple voltages at the wire joints do not, so they cancel. This gives 40 pairs per second.
2. **Reject spikes.** Keep the median of the last 5 pairs (`std::nth_element`). A plate knock affects one or two pairs, not three.
3. **Gate on stillness.** The bar is still when, over the last 1.0 s, every motion sample has $| |a| - 1000\ \text{mg} | < 30\ \text{mg}$ and $|\omega| < 3\ °/s$. Any sample over 8 g (a drop or a hard rack) invalidates the next 1.0 s.
4. **Average the still window.** Average the 40 medians from that 1.0 s window into $\bar{d}$. Divide by $\cos\theta$, where θ is the bar's tilt taken from the gravity vector.
5. **Convert.** $kg = slope \times (\bar{d} - zero - k_{temp} \times (temp_c - t_{ref}))$. The `zero` is captured with the empty bar racked, so `kg` is the load of the plates alone. Without a thermistor, the temperature term is left out.
6. **Report.** Send a `WeightEvent` on the first still window after the bar moves, or when the value changes by 0.2 kg or more. Racked counts as still: the J-hooks are inboard of the gauges, so a racked bar reads its full plate weight.

3. Complexity at realistic size. Each sample costs constant work: a subtraction, a 5-element median, a running sum and a few comparisons, about 2,000 CPU cycles per pair. At 40 pairs/s that is about 80,000 cycles per second, roughly 0.1% of the 64 MHz nRF52840. Memory is fixed at about 200 bytes (40 window values, 5 median

values, counters). At 100 times the rate (8,000 samples/s), A1 would use about 13% of the CPU, which still does not matter. The real limit is the HX711, which cannot sample faster than 80/s. Weight latency is set by the 1 s stillness window the requirement asks for, not by computation.

4. Why this one rather than the alternatives

Alternative	Why we did not choose it
Plain moving average, no reversal	Our SPICE drift case showed a 2.44 kg error without reversal and 0.03 kg with it. That alone breaks AC-01.1's ±1.0 kg.
Kalman filter on the raw stream	It needs a model of how load changes, which is a step function, not a smooth process. The extra tuning gives little over a gated average for static weighing and is harder to verify.
Weigh during motion as shear ÷ (g + a)	Works in principle, but a 1–2% accelerometer scale error becomes 1–2 kg at 100 kg. We keep it as a cross-check for later, not the primary reading.
Stream raw counts and compute on the phone	Rejected in D0 (DL-5): about 100 times more radio time, and the set is lost if the link drops.

5. Edge cases

Case	Behavior	Ties to
No input: HX711 never signals data-ready	After 200 ms, power-cycle C1 up to 3 times, then set `SENSOR_FAULT` and stop sending weight.	I2
Bar never still before the set	No weight event yet. C4 shows "hold the bar still for 1 s". The rack after the set is still, so the weight is read then.	UC-01 step 4
Deadlift: plates resting on the floor	Shear is about 0 because the floor carries the load. Windows where both sides read under 1.0 kg are labeled "floor" and not reported as a weight. The weight is taken from a lockout pause of at least 1 s.	AC-01.1
Repeated identical readings (duplicates)	Not re-sent unless the value changes by 0.2 kg or more.	I6
Ties	The median uses an odd window of 5, so there are no ties to break.	—
Drop or hard rack over 8 g	That window is discarded and the next 1 s is ignored.	NFR durability (D2)
Count at $\pm(2^{23} - 1)$ for 3 samples in a row	`OVER_RANGE`; no weight sent.	I2
Reading below −0.5 kg	Report 0.0 kg and set `ZERO_DRIFT`, so C4 asks for a re-zero on an empty bar.	AC-01.1
Sides differ by more than 2.5 kg	A1 reports each side honestly; C4 raises the imbalance alert.	AC-01.5
No thermistor (sprint build)	The temperature term is skipped; reversal still removes most drift.	—

A2. Rep count and bar speed (C2)

1. What it is and the problem it solves. A2 counts reps and measures each rep's mean and peak upward (concentric) velocity from the ring's motion sensor. A wrong count breaks AC-01.1 (5 reps); wrong speeds break US-02, because a coach adjusts load from them.

2. Inputs and outputs

Direction	Name	Exact type	Source and rate
In	`accel`	`int16[3]`, mg	I3, 104 samples/s
In	`gyro`	`int16[3]`, mdps	I3, 104 samples/s
In	`still`	`bool`	Stillness flag shared with A1, step 3
Out	`RepEvent`	`{rep_no: uint8 1–100, v_mean: float16 m/s, v_peak: float16 m/s, set_seq: uint16}`	I6 notify, once per rep
Out	`SetEnd`	`{set_seq: uint16, rep_count: uint8}`	I6 notify, 10 s after the last rep

Steps

1. **Find "up".** While the bar is still, the average accelerometer vector is gravity; store its direction ĝ. Between still windows, rotate ĝ with the gyro so bar roll does not tilt "up".
2. **Vertical acceleration.** `a_v = (a · ĝ − 1 g) × 9.80665 m/s²`.
3. **Velocity.** Integrate `a_v` with the trapezoid rule (dt = 1/104 s). Reset `v = 0` in every still window (a zero-velocity update), which removes integration drift at the top, the bottom pause and the rack.
4. **Phases with hysteresis.** DOWN when `v < −0.05 m/s` for at least 100 ms; UP when `v > +0.05 m/s` for at least 100 ms. A rep is an UP phase whose displacement `∫v dt` is at least 0.15 m. It may follow a DOWN phase (squat, bench) or start from still (deadlift). The rep closes when `v` drops back below +0.05 m/s.
5. **Speeds.** `v_mean = displacement ÷ UP duration`; `v_peak = max v` during UP.
6. **Drift guard.** If 8 s pass with no still window, apply a 2 s high-pass leak to `v`, and mark that rep's speed as low confidence.
7. **Set end.** After 10 s with no new rep and the bar still, send `SetEnd`.

3. Complexity at realistic size. Each motion sample costs constant work: one quaternion update, a dot product and an integration step, about 150 floating-point operations. At 104 samples/s that is about 16,000 operations per second, well under 1% of the CPU. Each rep keeps a few running values (start time, displacement, max), so memory does not grow with the length of the set. At 100 times the rate (10,400 samples/s), A2 would use a few percent of the CPU, and the motion sensor tops out at 6,660 samples/s anyway, so the difference does not matter. Rep and speed are known when the rep ends, well inside the 13 s summary budget (AC-01.1).

4. Why this one rather than the alternatives

Alternative	Why we did not choose it
Count peaks in raw acceleration magnitude	Gives no velocity, and a bounce at the bottom often counts twice.
Frequency analysis (FFT) of the motion	Needs several reps before it answers, cannot handle paused reps, and gives no per-rep speed.
Machine-learning classifier	Needs labeled training data we do not have, and its errors are hard to explain or fix.
Count reps from the strain signal	The strain changes with acceleration, but it is the weight channel; coupling the two makes both harder to debug. Kept as a possible cross-check.
Linear position transducer (string encoder)	The accepted reference for bar speed, but it is extra hardware tethered to the floor. We will use one to validate A2, not in the product.

5. Edge cases

Case	Behavior	Ties to
No input: motion sensor stops answering	Reset it once; if it still fails, send `REPS_UNAVAILABLE` and keep sending weight.	I3
Zero reps: bar lifted and put back	No `RepEvent`, so no set is created.	UC-01
Partial rep under 0.15 m	Not counted; the lifter can add it back by editing.	UC-01 A2
Pause at the bottom	The pause is a still window, so velocity resets; the following UP counts normally.	—
Failed rep: down, then re-racked without coming up 0.15 m	Not counted.	AC-01.1
Walk-out and re-rack steps	Mostly horizontal with small vertical travel; under 0.15 m, so not counted.	—
Touch-and-go deadlifts with no still window	The drift guard leaks velocity; speeds are flagged low confidence.	US-02
Bar roll or spin	The gyro rotates $\hat{g}$, so "up" stays up.	—
Duplicates after a link drop	`rep_no` is monotonic within `set_seq`; C4 drops repeats.	I6, AC-01.4
Ties	Not applicable; a rep is a single closed interval.	—
A set of more than 30 reps	Reps past 30 are counted but their speeds are not stored in the ring buffer.	I6

4. Build-versus-reuse decisions

Each candidate library was checked for maturity (active and used), license, performance on our hardware, and fit. "Build" means our code.

Piece (component)	Build or reuse	Library or service	License (SPDX)	One-line reason
A1 side-weight pipeline (C2)	Build	—	—	Specific to this product; nothing off the shelf combines reversal, stillness gating and tilt correction.
A2 rep and speed detector (C2)	Build	—	—	No maintained open-source library does barbell velocity from a single IMU.
HX711 read with polarity sync (C2)	Build, thin	Read timing checked against `bogde/HX711`	MIT (reference only)	The library is mature, but it cannot flip excitation between samples; our driver is about 60 lines.
Motion sensor driver (C2)	Reuse	Seeed_Arduino_LSM6DS3	MIT	Maintained by the board vendor for this exact chip.
Bluetooth LE stack and GATT (C2)	Reuse	Bluefruit nRF52 library in the Adafruit/Seeed nRF52 Arduino core, with Nordic's SoftDevice S140	LGPL-2.1-or-later (core); Nordic license (SoftDevice)	Mature, power-managed, and the board's default stack; writing a BLE stack is out of the question.
NFC tag driver (C2/C3)	Reuse	SparkFun ST25DV64KC Arduino Library	MIT	Written for the exact breakout we bought.
Flash file storage for the set buffer (C2)	Reuse	Adafruit LittleFS (in the core)	BSD-3-Clause (LittleFS)	Wear-levelled and power-loss safe, which matters for buffered sets.
Median and sort (C2)	Reuse	`std::nth_element` from the C++ standard library	GPL-3.0 with GCC Runtime Library Exception	Do not hand-roll sorting.
HTTP API framework (C5)	Reuse	FastAPI	MIT	Mature, async, generates the OpenAPI document for our contract.
Request validation (C5)	Reuse	Pydantic v2	MIT	Ranges and types from section 5 become declared models, not hand checks.
Database access and migrations (C5)	Reuse	SQLAlchemy 2 and Alembic	MIT	Standard Python tools; migrations keep schema changes reviewable.
PostgreSQL driver (C5)	Reuse	psycopg 3	LGPL-3.0-only	Used unmodified as a library, so LGPL places no obligation on our code.
Sign-in token verification (C5)	Reuse	PyJWT with its JWKS client	MIT	Do not hand-roll authentication.
Date and time handling (C5)	Reuse	Python `datetime` with PostgreSQL `timestamptz`	PSF-2.0 / PostgreSQL	Do not hand-roll date parsing; Pydantic parses ISO-8601.
Session arbitration rules (C5)	Build	On top of the database constraint	—	The "one open session, 12 s handover" rule is ours (UC-02).

Piece (component)	Build or reuse	Library or service	License (SPDX)	One-line reason
Duplicate-set handling (C5)	Build	`INSERT … ON CONFLICT DO NOTHING`	—	A few lines on top of the S8 key.
Database (C5)	Reuse	PostgreSQL 16, managed by Render	PostgreSQL	See section 6.
Identity (C5)	Reuse	Campus or Google sign-in over OpenID Connect (X3)	Provider terms of service	No passwords stored (DL-7).

5. API contract

5.1 C5 Account service, REST over HTTPS (interfaces I9 and I11)

All requests carry `Authorization: Bearer <id_token>` from X3. Bodies are JSON in UTF-8. Times are ISO-8601 UTC strings. Errors share one shape: `{"error": {"code": string, "message": string, "fields": [string]?}}`.

Endpoint	Inputs (type, required, valid range)	Output (status, shape, units)	Errors (status · code · when)
`POST /v1/sessions` (open or resume)	`barId` string, required, 6 chars `[A-Z0-9]` · `nonce` string, required, 8 hex digits · `busy` boolean, required (from the tag)	`201` new, or `200` resumed: `{sessionId: integer, barId: string, resumed: boolean, startedAt: string, barKg: number kg, rings: [{side: "L"|"R", ringId: string, calDate: string date}]}`	`400 INVALID_INPUT` bad field · `401 TOKEN_INVALID` or `TOKEN_EXPIRED` · `404 BAR_NOT_REGISTERED` · `409 BAR_IN_USE` another lifter's session is open and `busy` is true; body adds `retryAfterSec: integer` (AC-02.2) · `503 DB_UNAVAILABLE`
`POST /v1/sets` (store a confirmed set; idempotent)	`setId` string, required, `^[A-Z0-9]{6}-\d{1,10}-\d{1,5}$` · `sessionId` integer, required · `setSeq` integer, required, 0–65535 · `leftKg`, `rightKg` number, required, 0–300 kg · `totalKg` number, required, must equal left + right + barKg within 0.05 · `repsMeasured` integer, required, 1–100 · `repsSaved` integer, required, 0–100 · `reps` array, required, 1–100 items `{n: integer, vMean: number 0–5 m/s, vPeak: number 0–8 m/s}` · `confirmMode` string, required, `user` or `auto` · `startedAt`, `endedAt` string, required	`201 {setId: string, storedAt: string, duplicate: false}` · `200` same shape with `duplicate: true` when the `setId` already exists (AC-01.4)	`400 INVALID_INPUT` · `401` · `403 NOT_SESSION_OWNER` · `404 SESSION_NOT_FOUND` · `409 SET_AFTER_SESSION_END` the set started after its session ended · `422 TOTAL_MISMATCH` · `503 DB_UNAVAILABLE`
`POST /v1/sessions/{sessionId}/end`	`sessionId` path integer, required · `reason` string, required, `user` or `idle`	`200 {sessionId: integer, endedAt: string, setsSaved: integer, alreadyEnded: boolean}`	`401` · `403 NOT_SESSION_OWNER` · `404 SESSION_NOT_FOUND` · `503`. Ending an ended session is not an error: `200` with `alreadyEnded: true`.
`POST /v1/rings/{ringId}/status`	`ringId` path string, required · `batteryPct` integer, required, 0–100 · `faults` integer, required, 0–255 bit mask · `firmware` string, required, ≤ 16 chars · `reportedAt` string, required	`202 {}`	`400` · `401` · `404 RING_NOT_REGISTERED` · `503`
`GET /v1/me/sets` (lifter history)	`from`, `to` string query, optional, default last 30 days, range ≤ 366 days · `limit` integer, optional, 1–200, default 50 · `cursor` string, optional	`200 {sets: [{setId, endedAt, totalKg: number kg, repsSaved: integer, reps: [{n, vMean m/s, vPeak m/s}]}], nextCursor: string|null}`	`400 INVALID_RANGE` · `401` · `503`

Endpoint	Inputs (type, required, valid range)	Output (status, shape, units)	Errors (status · code · when)
`GET /v1/teams/{teamId}/sets` (coach view, I11)	`teamId` path integer, required · `from`, `to` string query, required, range ≤ 31 days	`200 {athletes: [{accountId: integer, name: string, sets: [{setId, endedAt, totalKg kg, reps: [{n, vMean m/s, vPeak m/s}]}]}]}`	`400 INVALID_RANGE` · `401` · `403 ROLE_NOT_ALLOWED` caller is not a coach of this team · `404 TEAM_NOT_FOUND` · `503`

Example request and response

```
POST /v1/sessions HTTP/1.1
Host: api.smartbarbell.example
Authorization: Bearer eyJhbGciOiJSUzI1NiIsImtpZCI6Ij…
Content-Type: application/json

{"barId": "R7Q2KX", "nonce": "1A2B3C4D", "busy": false}
```

```
HTTP/1.1 201 Created
Content-Type: application/json

{"sessionId": 41207, "barId": "R7Q2KX", "resumed": false,
 "startedAt": "2026-11-03T18:31:52Z", "barKg": 20.0,
 "rings": [{"side": "L", "ringId": "D4:3A:91:0C:22:7E", "calDate": "2026-10-28"},
           {"side": "R", "ringId": "C1:77:5B:E2:09:41", "calDate": "2026-10-28"}]}
```

Error example, a second lifter tapping 5 s after the first lifter racked (AC-02.2):

```
HTTP/1.1 409 Conflict
{"error": {"code": "BAR_IN_USE", "message": "This bar is still in use. Tap again in a few seconds."},
 "retryAfterSec": 7}
```

5.2 C2 Ring firmware, Bluetooth LE GATT (interface I6)

One custom service per ring with three characteristics. All packets are little-endian, type byte first, and fit one default-size notification (≤ 20 bytes).

Characteristic	Direction	Message (type byte) and fields	Errors
`INFO` (read)	C2 → C4	`protocol: uint8` (currently 1) · `side: char` · `ring_id: uint8[6]` · `fw: uint16` · `cal_date: uint16` days since 2020-01-01	—
`EVENTS` (notify)	C2 → C4	`0x01 WEIGHT {side: char, kg: float32, still: uint8, set_seq: uint16}` · `0x02 REP {rep_no: uint8, v_mean: float16 m/s, v_peak: float16 m/s, set_seq: uint16}` · `0x03 SET_END {set_seq: uint16, rep_count: uint8}` · `0x04 STATUS {battery_pct: uint8, faults: uint8, stored_sets: uint8}` · `0x05 STORED_SET {session_id: uint32, set_seq: uint16, kg: float32, rep_count: uint8, age_s: uint32}`	`0x7F ERROR {cmd: uint8, code: uint8}` : `1 BAD_NONCE`, `2 NOT_BOUND`, `3 BUSY_MEASURING`, `4 SENSOR_FAULT`, `5 BAD_LENGTH`

Characteristic	Direction	Message (type byte) and fields	Errors
`COMMANDS` (write with response)	C4 → C2	`0x81 ZERO {}` (only when the bar is still and racked) · `0x82 BIND {session_id: uint32, nonce: uint32}` · `0x83 ACK {set_seq: uint16}`	Answered with `0x7F ERROR` on failure; a successful `BIND` is answered with `STATUS`.

5.3 Versioning

The REST API is versioned by its path (`/v1`) and the Bluetooth protocol by the `protocol` byte in `INFO`. Removing or renaming a field, changing a type or a unit, adding a required input, or changing what an error code means is a breaking change and gets a new version, with the old one kept until the end of the term. Adding optional inputs, new response fields or new event types is not breaking. C4 refuses to bind a ring whose `protocol` is newer than it understands and asks the lifter to update the app.

6. Technology choices with justification

Statements about our own experience are marked ⟨confirm⟩ and must be checked by the team before submission.

Ring hardware platform and firmware: Seeed XIAO nRF52840 Sense, C++ on the Arduino nRF52 core. *Team skill fit:* Tyler owns the ring and has written Arduino-style C++ for the sprint prototypes ⟨confirm: prior embedded C/C++ coursework or projects⟩. The Arduino API keeps the learning curve to the BLE and power-management parts that are new to us. *Licensing:* the core is LGPL-2.1-or-later, used unmodified, so our firmware can stay under our own license; Nordic's SoftDevice license allows free use on Nordic chips; the driver libraries are MIT. *Community support:* the nRF52840 is one of the most widely used BLE chips; Seeed and Adafruit both maintain the core and their forums are active. *Performance:* a 64 MHz Cortex-M4F with a floating-point unit runs A1 and A2 in under 1% of its time; the board sleeps at tens of µA with wake-on-motion, which the weeks-per-charge target needs (DL-8). It includes the motion sensor and a USB-C LiPo charger, so C1 is the only custom board. *Cost and hosting:* $15.90 per board; no hosting. *Alternative passed over:* an ESP32-S3 board. It has Wi-Fi we do not want (D0 rejected Wi-Fi) and a higher sleep current. *Second alternative:* Zephyr with Nordic's nRF Connect SDK (Apache-2.0) gives finer power control but has a much steeper learning curve for a two-person team; we can move to it later because the hardware is the same.

Phone app (C4): native Android app in Kotlin with Nordic's Android-BLE-Library. *Team skill fit:* Colin owns C4 ⟨confirm: Kotlin or Java and Android experience; if none, plan two weeks of ramp-up in the Week 8 timeline⟩. Kotlin is close to Java, which we both used in coursework ⟨confirm⟩. *Licensing:* Kotlin and the Android SDK are Apache-2.0; Nordic's Android-BLE-Library is BSD-3-Clause; AppAuth-Android (sign-in) is Apache-2.0. *Community support:* Android BLE and NFC are documented by Google, and Nordic's library is the most used BLE wrapper on Android. *Performance:* the NFC tag's URL is registered as an Android App Link, so a tap opens the app directly, and the app reconnects to rings it has seen before without a device picker. That is what makes the 5 s connection target (AC-01.2) realistic. *Cost and hosting:* free to build and sideload; a Play Store developer account is a one-time fee if we publish. *Alternative passed over:* a web app using Web Bluetooth in Chrome, which D0 assumed. On Android it needs the device-picker dialog on every new page, and remembering devices without the picker (`getDevices()`) is still behind an experimental flag. We estimate 6–10 s from tap to connected through the picker ⟨confirm: measure on the sprint test page⟩, so the web app would likely fail AC-01.2. We keep the Web Bluetooth page as an engineering test tool only. We also give up iOS for this term.

Backend (C5): Python 3.12 with FastAPI. *Team skill fit:* both of us have written Python in coursework ⟨confirm: prior web or REST work⟩, and Colin owns C5. *Licensing:* Python (PSF-2.0), FastAPI, Pydantic, SQLAlchemy, Alembic and PyJWT are MIT; psycopg 3 is LGPL-3.0, used unmodified. *Community support:* FastAPI is one of the most used Python web frameworks, with thorough documentation. *Performance:* our load is a few requests per set, at most a few per second at peak; one small async instance handles hundreds of requests per second, so the 10 s save budget (AC-01.1) is dominated by network time. *Cost and hosting:* runs on the $7/month Render instance below. *Alternative passed over:* Node.js with Express. It is equally capable, but we would split our attention across two languages for no gain, and Pydantic's validation maps directly onto our API ranges.

Database: PostgreSQL 16. *Team skill fit:* we both took the databases course and used SQL there ⟨confirm⟩. *Licensing:* the PostgreSQL License is permissive. *Community support:* PostgreSQL is the most widely supported open-source relational database, and every host offers it managed. *Performance:* at 0.5 million sets a year every query in section 5 uses an index; even at 100 times that size the lookups stay logarithmic. The partial unique index enforces US-04 inside the database. *Cost and hosting:* a managed Render Postgres instance at $6/month plus $0.30 per GB of storage. *Alternative passed over:* Firebase Firestore. It is free at our size, but it has no partial unique index, so the one-open-session rule would move into application code (section 2). SQLite was also passed over: it cannot be shared safely by a hosted service that may run more than one copy.

Staff dashboard (C6): server-rendered pages from the same FastAPI app (Jinja2), with Chart.js for velocity charts. *Team skill fit:* HTML and a template language are familiar to both of us ⟨confirm⟩, and nothing new has to be deployed. *Licensing:* Jinja2 is BSD-3-Clause; Chart.js is MIT. *Community support:* both are long-established and widely used. *Performance:* the pages are read a few times a day by a handful of staff, so server rendering is fast enough and there is no client build.

Cost and hosting: served by the same web service at no extra cost. *Alternative passed over:* a React single-page app. It would add a second build, a second deployment and token handling in the browser for two simple read-only pages.

Hosting: Render (managed web service and managed PostgreSQL). *Team skill fit:* deploying from a Git push needs no server administration, which neither of us wants to spend the term on ⟨confirm: any prior Render, Heroku or similar use⟩. *Licensing:* commercial service; no license obligations on our code. *Community support:* well documented with an active community forum. *Performance:* the paid web instance stays running. The free tier is not usable for us: free web services sleep after 15 minutes without traffic and take about a minute to wake, which would break the 10 s save budget (AC-01.1), and free databases expire after 30 days. *Cost and hosting:* $0 workspace plan, $7/month web service, $6/month database plus storage, about $14/month or under $60 for the rest of the year. *Alternative passed over:* a university virtual machine. It is free, but we would run backups, TLS certificates and security updates ourselves.